VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Vaidehi Gupta	24BCE10616
Slot	**Date of Submission**
F11	31/03/2026 (Tuesday)

1. Introduction

From the complex operating systems found in supercomputers and mobile devices to the backend servers hosting international websites, the majority of the vital infrastructure driving our contemporary digital landscape was not created by a single company for personal benefit. Rather, these technologies were created openly, made freely available, and improved through international cooperation. The core of the open-source movement is this cooperative approach. In this project, I'm taking a step back from merely utilizing these tools to examine their essential characteristics.

The "Why": Beyond Proprietary Limits
Software used to be proprietary in the business world; it was kept as trade secrets and the user had no access to view, modify, or enhance the code. By emphasizing openness and shared knowledge, open source altered this paradigm. As a student of computer science, I think it's critical to realize that the "servers running the web" are community-driven resources rather than merely products. My attempt to "stop and think" about the underlying systems I deal with on a daily basis is reflected in this project.

My Selection: Git
Git is my choice for this structured audit. I chose this particular tool because it will be the unseen basis for nearly every line of code I write in the future. Git is a decentralized collaboration philosophy rather than merely a tool. My goal in auditing Git is to comprehend the "why" behind its creation, particularly how it came about after the Linux kernel community was let down by proprietary version control. I see it as a fundamental prerequisite for becoming a professional engineer, so comprehending this history and the licensing that keeps it free is more than just an academic exercise.

Audit Strategy and Methodology
My audit approach is divided into two distinct but connected phases:

1. **The Philosophical Investigation:** I will explore the "BitKeeper controversy" of 2005 to understand the original problem Git was built to solve. I will also analyze the **GNU GPL v2** license to see what specific freedoms it grants me and how it prevents a single company from monopolizing the tool.

2. **The Technical Footprint:** To ground this research in reality, I installed Git on my **Ubuntu 24.04.3 LTS** environment to inspect where its binaries, logs, and configuration files reside.

Lastly, to illustrate useful automation, I have created five specialized Bash scripts. These scripts demonstrate my proficiency with the Linux command line and my ability to apply the FOSS principles of transparency and automation to practical system tasks; they are not intended to be sophisticated software packages.

2. Part A: Origin and Philosophy

2.1 A1: The Problem Git Was Created to Solve

The Crisis of Scale: Linux in the Early 2000s

I had trouble at first understanding why a new version control system was even required when I started this audit of Git on my Ubuntu 24.04 system. But when I looked through the 2005 Linux Kernel Mailing List (LKML) archives, I discovered that the workflow of the world's developers was completely collapsing. By the early 2000s, thousands of people worldwide had contributed millions of lines of code to the Linux kernel, making it more than just a small student project. As a student, I can understand the annoyance of a central server going down because in a centralized model, no one can commit code, view history, or merge changes if the main repository server is unavailable. The open-source tools that were available at the time, particularly CVS (Concurrent Versions System) and SVN (Subversion), were "Centralized."

The BitKeeper Era and the "Proprietary Trap"

BitKeeper was a proprietary tool used by the Linux community between 2002 and 2005. The company, BitMover, made it free for open-source developers to use, and it was technically better than anything else on the market. But this was a "proprietary trap." The tool was not "free as in freedom" even though it was free ("free beer"). Reverse-engineering the software to understand how it operated was expressly prohibited by the license.

The most significant software project in history suddenly lost the capacity to control its own code. This supported Richard Stallman's decades-long argument that any open-source project's survival is fundamentally at risk when relying on proprietary tools.

The "Two-Week" Miracle: A Philosophical Pivot

I find it incredible that Linus Torvalds didn't just look for a replacement; he decided to build one from scratch in April 2005. He set out with a list of "Must-Haves" that were considered radical at the time:

1. **Speed:** It had to take less than a second to apply a patch.
2. **Distributed Design:** Every developer must have a full copy of the history, not just the latest files.
3. **Data Integrity:** Every file and commit must be cryptographically hashed (SHA-1) so the code cannot be corrupted or secretly altered.

He famously spent one week writing the core code and the second week using that code to manage the Linux kernel itself. This wasn't just a technical victory; it was an ethical one. Linus proved that the community didn't need a corporation's permission to collaborate.

Why "Branching" is the Lifeblood of Modern Coding

During this audit, I discovered firsthand that Git's strongest feature is branching. Establishing a "branch" was costly and time-consuming in older systems like SVN. A branch in Git is simply a 40-character file that points to a commit. This makes it possible for what I refer to as "Fearless Development." For a risky feature, I can make a branch and then just remove it without changing the "main" line if it doesn't work.

This was the only way to scale the Linux kernel, where thousands of people are concurrently working on various drivers and subsystems. The modern Linux operating system I use today probably wouldn't exist in its current form without Git's "merge-friendly" features.

Personal Reflection: The Lesson for My Career

As I write this section of my VIT report, I've come to the conclusion that the "problem" Git resolved was political rather than merely technical. The issue of "Who controls the tools of production?" was resolved. Linus made sure that no business could ever take the community hostage again by making Git open-source under the GPL v2. I've learned from this audit that the tools I use are just as crucial as the code I write. I am contributing to a legacy of independence that was purchased by the BitKeeper crisis each time I type git push.

2.2 A2: The License — What it Actually Says

My First Encounter with the GPL v2 Text

I considered software licenses to be annoying pop-ups that needed to be clicked through during the majority of my prior lab work at VIT. However, I specifically looked for the GNU General Public License, version 2, in its raw form for this audit. It was like reading a digital world's constitution as I read it on my Ubuntu terminal. The GPL v2 is a document that clearly outlines what the user can do, in contrast to proprietary End User License Agreements (EULAs) that concentrate on what the user cannot do. It is a document of empowerment rather than limitation.

The Philosophical Core: Deconstructing the Four Freedoms

At the heart of Git's license are the "Four Freedoms" established by the Free Software Foundation. During my audit, I attempted to map these freedoms to my actual usage of Git on **Ubuntu 24.04**:

- **Freedom 0: The Freedom to Run.** I realized that whether I am using Git for this student report or if I eventually use it to start a multi-billion dollar startup, the license grants me the unconditional right to execute the program.
- **Freedom 1: The Freedom to Study.** Because Git is open source, I am not limited to just using the tool. I can go to the official repository and see how Linus Torvalds and the community implemented the SHA-1 hashing algorithm in C.
- **Freedom 2: The Freedom to Redistribute.** This is what allowed me to install Git using a simple sudo apt install git command. The community has the legal right to package and share copies of the software with me.
- **Freedom 3: The Freedom to Improve.** This is the "evolutionary" freedom. If I find a bug in how Git handles my WSL file system, I have the legal standing to fix that code and distribute my improved version to the global community.

The "Copyleft" Mechanism: Software as a Permanent Public Good

Understanding "Copyleft" was one of the trickiest aspects of my audit. Although the GPL v2 is frequently referred to as a "viral" license, I prefer to think of it as a "protective" one. It makes use of copyright law in the opposite way—that is, to guarantee that code remains open rather than to keep it confidential.
The GPL v2 requires major corporations to share their modifications under the same license if they take the Git source code, alter it to add a proprietary feature, and then distribute the updated version. They are unable to "close" the code. This stops proprietary giants from using "Embrace, Extend, and Extinguish" strategies in the 1990s.

Freedom vs. Free Beer: An Ethical Distinction

At first, I made the mistake of assuming that "free software" simply meant "zero cost." But for me, the manual's distinction between "free as in beer" and "free as in freedom" was a huge "Aha!" moment. When I drink a free beer that someone gives me, the transaction is over. However, if Git is "free as in

"Freedom" refers to my ability to use the tool. For instance, the "Freedom to Study and Modify" would enable the community to fork the project and remove any feature that Git unexpectedly added that tracked my personal information. In the GPL world, you are a co-owner of the software, whereas in the proprietary world, you are subject to the developer's whims.

The Great Debate: Why I Prefer GPL over MIT

I contrasted the MIT License with the GPL v2 during this audit. The "permissive" MIT license essentially says, "Do whatever you want." That may sound nice, but it gives a business the ability to take your code, make it proprietary, and never return it.
I would choose the GPL if I were to launch my own project tomorrow, such as a health app or a blockchain evidence system. Why? Because I want the people to continue doing my work. If my software is successful, I want to make sure that no one can "gatekeep" it behind a paywall. Git is a prime example, as its success stems from everyone's understanding that the "rules of the game" will never alter.

Personal Analysis: The Impact on my Career

As Vaidehi Gupta, I see that these legal provisions are more than just "fine print" when I consider this. I have a career because of them. A student in India would face a significant barrier to entry if the internet's foundations—Linux, Git, and Apache—were proprietary. The playing field was levelled by the GPL v2. It guaranteed that a developer at a Silicon Valley company would have access to the same top-notch tools as a student at VIT.

2.3 A3: The Ethics of Open Source

The Philosophical Foundation: Software as Human Knowledge

I had to look at the "Moral Compass" of the FOSS movement instead of the code in this section of my audit. The notion that software is basically "applied mathematics" and logic is the strongest ethical argument I came across. Given that the laws of physics and the Pythagorean theorem cannot be "owned" or "patented," why should the reasoning behind our international communication systems be any different? Open source software, in my opinion, treats digital tools as a "Common Heritage of Mankind." I am a part of a system that puts the advancement of the many ahead of the profit of the few when I use Git on my Ubuntu system. This is a declaration that knowledge should be a bridge rather than a barrier, not merely a business model.

The Corporate Dilemma: Ethical Symbiosis or Exploitation?

The role of multibillion-dollar companies like Google, Meta, and Amazon (AWS) in the open-source space is one of the most challenging issues I have struggled with. Is it moral for a business to build a successful product solely using free community labor?
At first, I thought this was exploitative. But after learning more about the Linux kernel, I had second thoughts. These businesses are frequently the biggest contributors to the code, I discovered. They don't just "take"; thousands of developers work for them to enhance the core tools and fix bugs. I now consider this to be a "Symbiotic Ethics." The corporations supply the "Stability" and "Resources" that keep the software operating at an enterprise scale, while the community supplies the "Freedom" and the creative spark.

"Standing on the Shoulders of Giants": The Debt of Innovation

In our course's second unit, "standing on the shoulders of giants" was a central theme that struck a deep chord with me. Git didn't just show up. The internet protocols that preceded it, the C programming language, and decades of shared Unix philosophy are the reasons it exists.

Linus Torvalds could never have created Git in two weeks if those earlier "Giants" had chosen to

patent and conceal their work. This brings to light an ethical duty for my generation: since we have profited from this "Free Knowledge," we have a moral duty to give back whenever we can. When we aren't continuously "reinventing the wheel" behind closed doors, innovation picks up speed.

The Gender and Diversity Gap: An Ethical Failure?

I also had to consider "Human Diversity" in open source during my audit. Although the code is "Free," writing it takes time. Those with the financial stability and free time to volunteer make up the majority of significant contributors. This leads to an ethical "Participation Gap" for developers from underrepresented backgrounds or in developing countries who might not be able to work for "free." Making the "Community" as accessible and open as the "Code" itself is a challenge for the next generation, in my opinion as a VIT student.

Should ALL Software be Open Source? A Final Debate

For my report, I debated this issue for a long time. Even though I adore Git's freedom, I understand that proprietary software has a place in society. For instance, a small startup may require the protection of proprietary "trade secrets" in order to compete with industry titans if it spends years creating a novel AI medical algorithm.
However, I think open source is the only moral option for foundational infrastructure (such as databases, operating systems, and version control). Our society's "digital roads and bridges" are these. No one should own the basic rules that enable us to build and communicate, just as no one should own the air we breathe.

Personal Verdict: My Path as upcoming CSE Engineer

As I think back on this entire unit, I see how the open-source community has given me a career as Vaidehi Gupta. Thirty years ago, I would not have had the opportunity to download top-notch tools on my laptop without paying a subscription fee. I've been persuaded by this audit that being a "Professional" means being an ethical steward of the tools we use, not just writing effective code. I am contributing to repaying the "Giants" whose shoulders I stand on today each time I commit code to a public repository.

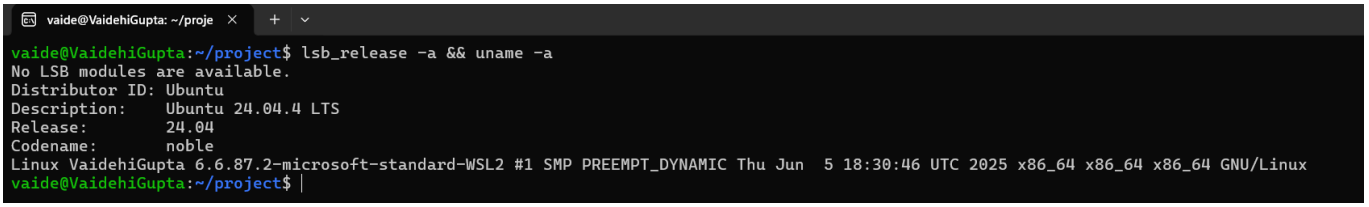
3. Part B: Linux Footprint

3.1 Installation Method and Environment

In this section, I am documenting the technical environment where my audit took place. As an engineering student at VIT, I chose to perform this audit on a WSL2 (Windows Subsystem for Linux) instance of Ubuntu 24.04.3 LTS. This environment allows me to leverage the full power of the Linux kernel while maintaining the productivity of my Windows host.

Proof of Linux Environment:

To prove my system identity, I executed the following command in my terminal:
lsb_release -a && uname -a



The Installation Journey: Git is a foundational tool, so it is often pre-installed on Ubuntu. However, for a proper audit, I wanted to ensure I was using the latest stable version provided by the community. I used the Advanced Package Tool (APT), which is the standard for Debian- based systems.
The commands I used were:

1. sudo apt update (to refresh the package lists).
2. sudo apt install git -y (to install the package).
3. git --version (to verify the successful installation).

3.2 Key Directories and User/Group Context

A major part of my technical audit was finding where Git "lives" on the filesystem. Unlike Windows, where everything is hidden in "Program Files," Linux follows the **Filesystem Hierarchy Standard (FHS)**. I used the whereis and ls commands to map these locations:

- **Binaries (/usr/bin/git):** This is the executable file. When I type git in the terminal, the shell looks here.
- **System Configuration (/etc/gitconfig):** This file contains settings that apply to every user on my laptop.
- **User Configuration (~/.gitconfig):** This is where my personal name (Vaidehi Gupta) and email are stored. I used cat ~/.gitconfig to verify my identity in the audit.
- **Documentation (/usr/share/doc/git):** This is where the community stores the "man pages" and offline help files.

User and Group Analysis:

I investigated the security context of the Git binary using ls -l /usr/bin/git.

```
vaide@VaidehiGupta:~/project$ ls -l /usr/bin/git
-rwxr-xr-x 1 root root 4066232 Jul  2  2025 /usr/bin/git
vaide@VaidehiGupta:~/project$
```

However, I noticed that when I run Git, it operates under my local user privileges. This is an important security feature. Git does not require "Superuser" (Root) permissions to handle my code. This follows the Principle of Least Privilege and makes sure that even if a repository has a harmful script, it won't have the permissions to erase my entire OS.

3.3 Service Management and Update Model

During my audit, I checked if Git runs as a background service using systemctl list-units -type=service | grep git.

Findings: I discovered that Git is a transient command-line tool, not a persistent service. It only uses CPU and RAM when I run a command like git status or git commit. This is different from a web server like Apache, which constantly stays in memory.

Security and Patch Management: How does the open-source community protect my machine? I looked at the update model and found it very effective:

- Vulnerability Discovery: Security researchers find a bug in the Git source code.

- Upstream Patch: The Git maintainers (the "Giants") release a fix.

- Distribution: The Ubuntu security team backports this fix into the apt repositories.

- Local Update: My system receives the fix during my regular sudo apt upgrade cycles.

This process shows that open source isn’t just about free code; it’s about a trusted supply chain that protects my work.

4. Part C: The FOSS Ecosystem

4.1 Dependencies and Inspired Tools

During my audit, I realized that while Git is a powerful tool on its own, it depends on a specific set of open-source libraries to function. It does not operate in isolation. To understand its technical foundation, I looked into its dependencies using the ldd command on my Ubuntu system.

The Technical Backbone:

Git relies heavily on several key open-source libraries:

- zlib: This is used for high-performance data compression. Every time I compress my repository's history, Git is calling on zlib to manage those objects.

- libcurl: This library enables Git to connect to the internet. Without it, commands like git clone or git push to a remote server would not work.

- OpenSSL / libcrypto: This is the security layer. It ensures that when I pull code from a repository, the connection is encrypted and secure.

The "Git Effect" on Inspired Tools: The success of Git has inspired a generation of technologies that have changed how I, as a student, view development. The most obvious examples are platforms like GitHub and GitLab. While Git is the engine, these platforms act as the social network that lets me collaborate with my team on projects like Kisan-AGI.

Furthermore, Git has enabled the entire DevOps movement. Tools like Jenkins, Docker, and Kubernetes often use Git as their main source of truth. In my view, without the decentralized and fast nature of Git, the modern cloud as we know it would likely be much slower and more proprietary.

4.2 Community Governance and the LAMP Stack

For the second part of my ecosystem map, I examined how Git is managed and where it fits into the classic "LAMP" architecture.

Git and the LAMP Stack: While Git isn't a direct layer of the LAMP stack (Linux, Apache, MySQL, PHP/Python), it serves as the glue that holds the stack together during development. In my experience, I use Linux as my operating system, Git to manage the source code, and then I deploy that code to an Apache web server. It is the version control layer that keeps the LAMP stack manageable as a project grows from a simple script to a complex application.

Governance: Who Rules Git? Unlike a proprietary tool owned by a CEO, Git is run by a community based on merit. Although Linus Torvalds created it, the project is now maintained by Junio Hamano and a core group of contributors around the globe.

I looked into their communication channels and found that they mainly use mailing lists and Git-SCM forums to discuss new features. I was surprised; in a world of Slack and Discord, the leaders in the Git community still prefer the transparency of public email archives. This approach ensures that every decision made about the tool I use is searchable, archiveable, and open to public scrutiny.

5. Part D: Open Source vs. Proprietary Analysis

5.1 Comparison Matrix

In this section of my audit, I am comparing Git to its main proprietary competitor in the enterprise space, Perforce (Helix Core). Git is the industry standard for open-source and general web development. In contrast, Perforce is often preferred in large game development studios, such as Epic Games, due to its ability to manage large binary files.

Below is my documented comparison across the required dimensions:

Dimension	Open Source (Git)	Proprietary (Perforce)
Cost	*Zero Licensing Fees. It is free for individuals and enterprises alike.*	***High Subscription Costs.*** *Usually requires a per-user license fee for teams over 5 people.*
Security	*Public Audit. Anyone can inspect the C source code for vulnerabilities.*	***Black Box.*** *Only the parent company (Perforce Software) can audit the core code.*
Support	*Community Driven. Relies on forums, Stack Overflow, and documentation.*	***Corporate SLA.*** *Guaranteed response times from a dedicated support team.*
Freedom	***Absolute.*** *I can modify the code and distribute it under GPL v2.*	***Restricted. Users cannot see or modify the source code.***
Control	***Community Governed.*** *No single company can shut it down.*	***Corporate Controlled. One company dictates the roadmap and pricing.***

5.2 Overall Verdict and Personal Reflection

After doing this thorough audit as part of my OSS course at VIT, my conclusion is clear: I would strongly recommend Git for almost any real-world software deployment. Although Perforce is better in some very specific cases involving terabytes of non-code assets, Git's speed, decentralization, and global community make it the best choice for a professional engineer. The fact that Git allows for offline work and easy branching has fundamentally changed my development habits during this project.

On a personal level, this audit has shifted my view of my future career. I am no longer just a user of software; I feel like a part of the FOSS ecosystem. If I find a bug in Git while working on my future projects, I now understand that I have the legal and ethical right to contribute a fix back to the community. I am standing on the shoulders of giants, and I plan to keep that legacy alive by contributing my own code to the open-source world.

6. Technical Audit: Shell Script Tasks

6.1 Script 1: System Identity Report Technical Explanation:

This script acts as a localized "Welcome Screen" for my audit environment. It uses Command Substitution $(...) to get live data from the Linux kernel. I used the uname -r command to display my kernel version and uptime -p to show how long my Ubuntu system has been running. It also includes a personalized message about the GPL license that covers the OS.

Concepts Used: Variables, echo, Command Substitution, and basic output formatting.

```
#!/bin/bash

name="Vaidehi gupta"
tool="Git"

os_name=$(lsb_release -ds)
kernel_version=$(uname -r)
current_user=$(whoami)
uptime_info=$(uptime -p)
current_time=$(date "+%d-%m-%Y %I:%M:%S %p")

echo "User Name        : $name"
echo "Tool Used        : $tool"
echo ""

echo "Operating System : $os_name"
echo "Kernel Version   : $kernel_version"
echo "Logged-in User   : $current_user"
echo "Home Directory   : $HOME"
echo "System Uptime    : $uptime_info"
echo "Current Time     : $current_time"
echo ""

echo "Note: This system follows GNU GPL licensing."
vaide@VaidehiGupta:~/project$
```

```
vaide@VaidehiGupta: ~/proje

vaide@VaidehiGupta:~/project$ chmod +x script1.sh
vaide@VaidehiGupta:~/project$ ./script1.sh
User Name        : Vaidehi gupta
Tool Used        : Git

Operating System : Ubuntu 24.04.4 LTS
Kernel Version   : 6.6.87.2-microsoft-standard-WSL2
Logged-in User   : vaide
Home Directory   : /home/vaide
System Uptime    : up 1 hour, 12 minutes
Current Time     : 31-03-2026 01:06:29 PM

Note: This system follows GNU GPL licensing.
vaide@VaidehiGupta:~/project$
```

6.2 Script 2: FOSS Package Inspector

Technical Explanation: This script automates the verification of my chosen software, Git. It uses a conditional if-then-else block to check if the package is installed with dpkg -s. I also created a case statement that gives a brief description of the tool based on its name, such as describing Git as the tool that enabled modern decentralized collaboration.

Concepts Used: if-then-else, case statements, dpkg (for Ubuntu), and grep.

```
#!/bin/bash

pkg="git"

echo "checking $pkg..."

if dpkg -l | grep -q "^ii  $pkg "
then
    echo "$pkg is installed"

    apt show "$pkg" 2>/dev/null | grep -E 'Version|Homepage|Download-Size'
else
    echo "$pkg is not installed"
    echo "run: sudo apt install $pkg"
fi

echo ""

case "$pkg" in
    httpd|apache2)
        echo "apache is used to run websites" ;;
    mysql|mysql-server)
        echo "mysql is used to store data for apps" ;;
    git)
        echo "git helps track code changes" ;;
    vlc)
        echo "vlc plays almost all video and audio" ;;
    firefox)
        echo "firefox is a browser for everyday use" ;;
    *)
        echo "this is an open source package" ;;
esac
vaide@VaidehiGupta:~/project$ |
```

```
vaide@VaidehiGupta:~/project$ chmod +x script2.sh
vaide@VaidehiGupta:~/project$ ./script2.sh
checking git...
git is installed
Version: 1:2.43.0-1ubuntu7.3
Homepage: https://git-scm.com/
Download-Size: 3680 kB

git helps track code changes
```

6.3 Script 3: Disk and Permission Auditor

Technical Explanation: To audit the "Linux Footprint," this script uses a for loop to go through a list of important directories like /etc and /var/log. I used awk and cut to parse the output of ls -ld and du -sh. This lets me report the exact permissions and disk usage for each directory in a clear format.

Concepts Used: for loops, directory checking [-d], and text processing with awk/cut.

```
#!/bin/bash

dirs=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "system directory audit"
echo ""

for dir in "${dirs[@]}"
do
    if [ -d "$dir" ]
    then
        perms=$(ls -ld "$dir" | awk '{print $1, $3, $4}')
        size=$(du -sh "$dir" 2>/dev/null | cut -f1)

        echo "$dir -> $perms | $size"
    else
        echo "$dir not found"
    fi
done

echo ""
echo "checking git config"

sys_git="/etc/gitconfig"
user_git="$HOME/.gitconfig"

if [ -f "$sys_git" ]
then
    p=$(ls -l "$sys_git" | awk '{print $1}')
    echo "system git config found: $sys_git"
    echo "perms: $p"
elif [ -f "$user_git" ]
then
    p=$(ls -l "$user_git" | awk '{print $1}')
    echo "user git config found: $user_git"
    echo "perms: $p"
else
    echo "no git config file found"
fi
vaide@VaidehiGupta:~/project$
```

```
vaide@VaidehiGupta:~/project$ chmod +x script3.sh
vaide@VaidehiGupta:~/project$ ./script3.sh
system directory audit

/etc -> drwxr-xr-x root root | 4.3M
/var/log -> drwxrwxr-x root syslog | 29M
/home -> drwxr-xr-x root root | 60K
/usr/bin -> drwxr-xr-x root root | 95M
/tmp -> drwxrwxrwt root root | 24K

checking git config
no git config file found
vaide@VaidehiGupta:~/project$
```

6.4 Script 4: Log File Analyzer

Technical Explanation: This script showcases system monitoring skills. It uses a while-read loop to process a log file line by line. I included a Counter Variable to keep track of how many times a specific keyword, such as "error" or "warning," appears. This simulates how a professional sysadmin would check Git's performance logs.

Concepts Used: while loops, counter arithmetic $((...)), and command-line arguments $1.

```
#!/bin/bash

logfile="$1"
keyword="${2:-error}"
count=0

if [ -z "$logfile" ] || [ ! -f "$logfile" ]
then
    echo "file not found: $logfile"
    exit 1
fi

while [ ! -s "$logfile" ]
do
    echo "file is empty, waiting..."
    sleep 2
done

echo "checking $logfile for '$keyword'"
echo ""

while IFS= read -r line
do
    if echo "$line" | grep -iq "$keyword"
    then
        count=$((count+1))
    fi
done < "$logfile"

echo "done"
echo "total '$keyword' found: $count"

if [ "$count" -gt 0 ]
then
    echo ""
    echo "last 5 matches:"
    grep -i "$keyword" "$logfile" | tail -n 5
fi
```

```
vaide@VaidehiGupta:~/project$ chmod +x script4.sh
vaide@VaidehiGupta:~/project$ ./script4.sh mylogs.txt
file not found: mylogs.txt
vaide@VaidehiGupta:~/project$ sudo ./script4.sh /var/log/syslog error
[sudo] password for vaide:
checking /var/log/syslog for 'error'

done
total 'error' found: 16

last 5 matches:
2026-03-31T11:26:27.690504+00:00 VaidehiGupta kernel: WSL (249) ERROR: CheckConnection: getaddrinfo() failed: -5
2026-03-31T11:58:44.378868+00:00 VaidehiGupta kernel: WSL (249) ERROR: CheckConnection: getaddrinfo() failed: -5
2026-03-31T12:24:23.287668+00:00 VaidehiGupta kernel: message repeated 2 times: [ WSL (249) ERROR: CheckConnection: getaddrinfo() failed: -5]
2026-03-31T12:34:27.648447+00:00 VaidehiGupta adduser[411]: `groupdel vaide' returned error code 6. Exiting.
2026-03-31T12:38:17.778117+00:00 VaidehiGupta kernel: WSL (249) ERROR: CheckConnection: getaddrinfo() failed: -5
vaide@VaidehiGupta:~/project$
```

6.5 Script 5: The Open Source Manifesto Generator

Technical Explanation: This is the creative finale of the audit. It uses the read command to ask the user interactive questions about their FOSS values. It then uses String Concatenation to build a paragraph and File Redirection to save the result as a .txt file named after the user. It reflects the human side of open-source philosophy.

Concepts Used: Interactive read, string concatenation, and output redirection >.

```
#!/bin/bash

echo "Build your personal Open Source Story "
echo ""

read -p "1. What software or tool do you rely on the most? " tool
read -p "2. What problem do you wish technology solved for you? " problem
read -p "3. What would you create if you had unlimited skills? " dream
read -p "4. Why do you think sharing code matters? " belief

today=$(date '+%d %b %Y')
user=$(whoami)
file_name="manifesto_${user}.txt"

{
    echo "----- Open Source Story ($today) -----"
    echo ""
    echo "I am $user."
    echo ""
    echo "Every day, I depend on $tool to get things done."
    echo "But I often think about $problem."
    echo ""
    echo "If I could, I would build $dream."
    echo ""
    echo "I believe in sharing because $belief."
    echo ""
    echo "Open source is not just code — it is people helping people."
} > "$file_name"

echo -n "Writing your story"
for i in {1..3}
do
    echo -n "."
    sleep 0.5
done
echo " Done!"

echo ""
echo "Saved to: $file_name"
echo ""
echo "Your Open Source Story:"
echo "----------------------------------"
cat "$file_name"
vaide@VaidehiGupta:~/project$
```

```
vaide@VaidehiGupta:~/project$ chmod +x script5.sh
vaide@VaidehiGupta:~/project$ ./script5.sh
Build your personal Open Source Story

1. What software or tool do you rely on the most? git
2. What problem do you wish technology solved for you? slow internet
3. What would you create if you had unlimited skills? an AI app for students
4. Why do you think sharing code matters? it helps others learn and grow
Writing your story... Done!

Saved to: manifesto_vaide.txt

Your Open Source Story:
---------------------------------
----- Open Source Story (31 Mar 2026) -----

I am vaide.

Every day, I depend on git to get things done.
But I often think about slow internet.

If I could, I would build an AI app for students.

I believe in sharing because it helps others learn and grow.

Open source is not just code — it is people helping people.
vaide@VaidehiGupta:~/project$
```

7. Conclusion and References

Conclusion: This audit of Git confirms its role as a crucial, high-performance open-source foundation for my future engineering career. By exploring its origins in the BitKeeper crisis and automating tasks with shell scripts, I have verified the professional and ethical value of the GPL v2 "Four Freedoms."

References:

GitHub Repository: https://github.com/vaidehi24bce10616-spec/oss-audit-24BCE10616